◆ CATCHFORM

Cahier des charges et dossier de projet

SaaS de réception de formulaires pour sites statiques

Projet	CatchForm — backend de formulaires pour sites statiques
Nature du document	Cahier des charges fonctionnel et technique, bilan de réalisation et feuille de route
Version	1.1
Date	16 octobre 2026
État du projet	MVP développé et validé en local (27/27 tests automatisés, 29/29 contrôles de recette) — non déployé
Porteur du projet	[à compléter]
Rédaction	Claude Code (assistant IA)
Diffusion	Interne — confidentiel

Historique des versions

Version	Date	Auteur	Modifications
1.0	16/10/2026	Claude Code	Création du document : cahier des charges, bilan du MVP, feuille de route.
1.1	16/10/2026	Claude Code	Tests exécutés (27/27 réussis), recette sur serveur réel (29/29), configuration Render vérifiée ; statuts, risques et feuille de route mis à jour.

Sommaire

1. Synthèse du projet 4
1.1 Chiffres clés 4
1.2 Prochaines étapes 4
2. Présentation du projet 5
2.1 Contexte 5
2.2 Problématique 5
2.3 Solution proposée 5
2.4 Objectifs 5
2.5 Périmètre 6
3. Analyse du marché 7
3.1 Cibles et personas 7
3.2 Concurrence 7
3.3 Positionnement et différenciation 7
4. Modèle économique 8
4.1 Offres 8
4.2 Hypothèses de revenus 8
4.3 Stratégie d'acquisition 8
5. Spécifications fonctionnelles 9
5.1 Acteurs 9
5.2 Parcours principal 9
5.3 Exigences par module 9
5.4 Récapitulatif 12
6. Exigences non fonctionnelles 13
6.1 Sécurité 13
6.2 Performance et disponibilité 13
6.3 Conformité RGPD 13
6.4 Ergonomie, compatibilité et maintenabilité 14
7. Architecture technique 15

7.1 Choix technologiques 15
7.2 Schéma de fonctionnement 15
7.3 Modèle de données 16
7.4 Routes de l'application 16
7.5 Configuration 17
7.6 Organisation du code 17
8. Bilan : ce qui a été réalisé 19
8.1 Déroulé des travaux 19
8.2 Livrables 19
8.3 Couverture des tests 20
8.4 Recette sur serveur réel 21
8.5 Limites connues du code actuel 21
9. Reste à faire — feuille de route 23
Vue d'ensemble 23
9.1 Phase 0 — Validation technique 23
9.2 Phase 1 — Mise en ligne de la bêta 23
9.3 Phase 2 — Prérequis au lancement commercial 24
9.4 Phase 3 — Croissance 24
10. Planning prévisionnel 26
Jalons de décision 26
11. Recette — critères d'acceptation du MVP 27
12. Budget prévisionnel 28
12.1 Phase bêta 28
12.2 Production 28
12.3 Coûts hors technique 28
13. Analyse des risques 29
14. Glossaire 30
Annexes 31
A. Code d'intégration type 31
B. Commandes utiles (Windows, PowerShell) 31

1. Synthèse du projet

CatchForm est un logiciel en ligne (SaaS) qui permet aux sites statiques de recevoir des formulaires sans développer de serveur. Le client crée un formulaire dans CatchForm, colle l'adresse obtenue dans l'attribut `action` de sa balise `<form>`, puis reçoit chaque message par e-mail et dans un tableau de bord.

Ce document constitue le cahier des charges du projet. Il décrit le besoin, le marché et le modèle économique, spécifie les exigences fonctionnelles et techniques, dresse le bilan de ce qui a été développé et détaille ce qui reste à faire jusqu'au lancement commercial, puis au-delà.

> **État d'avancement au 16 octobre 2026**
>
> Le produit minimum viable (MVP) est **entièrement codé et validé en local** : application web, interface, tests automatisés, configuration de déploiement et documentation.
>
> **Résultats :** 27 cas de test automatisés sur 27 réussis, et 29 contrôles de recette sur 29 réussis sur un serveur réellement lancé. Aucune correction n'a été nécessaire. La configuration Render a été vérifiée dans la documentation officielle.
>
> **Reste à valider :** le rendu visuel dans un navigateur, l'envoi réel d'un e-mail (clé Resend nécessaire) et le fonctionnement en production. **Prochaine étape :** mise en ligne de la bêta (phase 1).

1.1 Chiffres clés

62 exigences fonctionnelles spécifiées	**37** exigences développées et testées	**25** exigences à faire ou partielles
27 / 27 cas de test automatisés réussis	**29 / 29** contrôles de recette réussis	**0 €** coût de démarrage (offres gratuites)

1.2 Prochaines étapes

1. **Phase 0 — Validation technique** : terminée, à l'exception d'une vérification visuelle dans un navigateur.
2. **Phase 1 — Bêta en ligne** : mise en ligne gratuite sur Render, hébergement à Francfort (UE).
3. **Phase 2 — Lancement commercial** : paiement Stripe, comptes complets, sécurité renforcée, pages légales et RGPD, hébergement payant.
4. **Phase 3 — Croissance** : intégrations, anti-spam avancé, statistiques, offre Agence.

2. Présentation du projet

2.1 Contexte

Une part importante des sites web est aujourd'hui **statique** : pages HTML hébergées sur GitHub Pages, Netlify ou Cloudflare Pages, sites produits avec Hugo, Jekyll, Astro ou des outils sans code. Ces sites sont rapides, sûrs et peu coûteux à héberger, car aucun serveur applicatif ne tourne derrière.

Cette absence de serveur a une limite : **un site statique ne peut pas traiter un formulaire.** Quand un visiteur clique sur « Envoyer », il faut un programme côté serveur pour recevoir, stocker et transmettre les données.

2.2 Problématique

Un freelance qui livre un site vitrine, ou un développeur qui publie son portfolio, a besoin d'un simple formulaire de contact. Il a trois options, toutes insatisfaisantes :

- **Développer un backend** : coût de développement, serveur à payer, sécurité et maintenance à assumer pour un besoin minime.
- **Utiliser la fonction de son hébergeur** : il reste lié à cet hébergeur.
- **Utiliser un service spécialisé** : les solutions établies sont surtout anglophones et pas toujours hébergées dans l'Union européenne.

2.3 Solution proposée

CatchForm attribue à chaque formulaire une **adresse de réception (endpoint)** unique. Il suffit de la placer dans l'attribut `action` du formulaire HTML. CatchForm se charge du reste :

- réception et stockage des soumissions ;
- filtrage du spam ;
- notification par e-mail du propriétaire du site ;
- redirection du visiteur vers une page de remerciement ;
- consultation et export des données depuis un tableau de bord.

L'intégration ne demande **aucune ligne de code serveur** et prend moins de trois minutes.

2.4 Objectifs

Les cibles ci-dessous sont indicatives et doivent être validées par le porteur du projet.

Objectif	Indicateur	Cible indicative
Mettre en ligne une bêta	Service accessible en HTTPS, `/healthz` au vert	Fin de la semaine 1
Garantir une intégration rapide	Temps entre l'inscription et la première soumission	Moins de 3 minutes
Valider l'intérêt	Comptes créés	50 comptes, 3 mois après le lancement
Valider la monétisation	Clients Pro payants	10 clients (≈ 90 € de revenu mensuel), 6 mois après le lancement
Couvrir les coûts	Revenu mensuel ≥ coût d'hébergement	≈ 2 clients Pro

2.5 Périmètre

Inclus dans le MVP

- Comptes utilisateurs (inscription, connexion).
- Formulaires illimités avec endpoint unique.
- Réception HTML et JSON, CORS, anti-spam par champ piège.
- Notifications e-mail, redirection, page de remerciement.
- Tableau de bord, pagination, suppression, export CSV.
- Plans Gratuit et Pro avec quota mensuel.
- Site public (accueil, tarifs) et configuration de déploiement.

Hors MVP (versions ultérieures)

- Paiement en ligne automatisé (Stripe).
- Réinitialisation du mot de passe, vérification de l'e-mail, gestion du compte.
- Limitation de débit, anti-spam avancé.
- Intégrations (webhooks, Slack, Google Sheets), pièces jointes, statistiques.
- Back-office d'administration, offre Agence.

3. Analyse du marché

3.1 Cibles et personas

Persona	Profil	Besoin	Attentes envers CatchForm
Léa, freelance web	Livre 10 à 20 sites vitrines par an pour des artisans et des PME.	Un formulaire de contact fiable sur chaque site, sans serveur à maintenir.	Formulaires illimités, redirection personnalisée, e-mails, prix bas.
Karim, développeur	Blog sous Hugo, portfolio sur GitHub Pages.	Un formulaire de contact gratuit et simple.	Plan gratuit, envoi `fetch()` en JSON, intégration en quelques minutes.
Agence web	5 à 20 personnes, dizaines de sites clients.	Centraliser les formulaires de tous les clients, en conformité avec le RGPD.	Hébergement UE, export CSV ; à terme comptes d'équipe et marque blanche.

3.2 Concurrence

Solution	Type	Points forts	Limites exploitables
Formspree	Service spécialisé, leader historique	Notoriété, maturité, nombreuses intégrations	Interface et support en anglais
Netlify Forms	Fonction intégrée à un hébergeur	Aucune configuration pour les sites hébergés chez Netlify	Réservé aux sites hébergés chez Netlify
Basin, Getform, Web3Forms…	Services spécialisés	Offres variées, plans gratuits parfois généreux	Positionnement générique, peu tourné vers le marché francophone
Backend sur mesure	Développement interne	Contrôle total	Coût, maintenance et sécurité à assumer

Les offres et les tarifs des concurrents évoluent. Une étude tarifaire à jour doit être menée avant le lancement (voir phase 2).

3.3 Positionnement et différenciation

> **« Le backend de formulaires simple, en français, hébergé en Europe. »**

- **Langue** : interface, documentation et support en français.
- **Conformité** : données hébergées dans l'Union européenne (Francfort), atout RGPD pour les agences et leurs clients.
- **Prix** : plan Pro à 9 €/mois, positionné sous les offres des leaders (à confirmer par l'étude tarifaire).
- **Simplicité** : un copier-coller, aucune bibliothèque à installer.
- **Acquisition virale** : la mention « Propulsé par CatchForm » sur les pages de remerciement du plan gratuit fait connaître le service.

4. Modèle économique

4.1 Offres

Modèle **freemium** : un plan gratuit pour l'adoption, un plan payant pour la monétisation.

Caractéristique	Gratuit	Pro
Prix	0 €	9 € / mois
Soumissions	100 par mois, tous formulaires confondus	Illimitées
Formulaires	Illimités	Illimités
Notifications e-mail	Oui	Oui
Anti-spam, export CSV	Oui	Oui
Mention « Propulsé par CatchForm »	Oui	Non
Support	Standard	Prioritaire

Une **offre Agence** (comptes d'équipe, gestion multi-clients, marque blanche) est à étudier une fois les premiers clients Pro acquis.

4.2 Hypothèses de revenus

Clients Pro	Revenu mensuel récurrent	Revenu annuel
2	18 €	216 €
10	90 €	1 080 €
50	450 €	5 400 €
100	900 €	10 800 €

Montants hors taxes, avant frais de paiement, impôts et cotisations sociales. Les coûts de fonctionnement sont détaillés au chapitre 12.

4.3 Stratégie d'acquisition

- **Référencement naturel** : tutoriels ciblés, par exemple « formulaire de contact Hugo sans backend » ou « formulaire GitHub Pages ».
- **Communautés** : Reddit (r/webdev, r/SideProject), Indie Hackers, Product Hunt, communautés de développeurs francophones.
- **Démarchage** : freelances et agences web, qui livrent de nombreux sites vitrines chaque année.
- **Effet viral** : mention « Propulsé par CatchForm » sur chaque page de remerciement du plan gratuit.

5. Spécifications fonctionnelles

5.1 Acteurs

Acteur	Description
Prospect	Personne qui découvre CatchForm sur le site public (accueil, tarifs).
Client	Titulaire d'un compte CatchForm, propriétaire d'un ou de plusieurs formulaires.
Visiteur final	Personne qui remplit un formulaire sur le site d'un client. Il n'a pas de compte CatchForm.
Administrateur	Exploitant du service. Agit aujourd'hui en ligne de commande (`set-plan`), demain via un back-office.

5.2 Parcours principal

1. Le client crée un compte, gratuitement et sans carte bancaire.
2. Il crée un formulaire et lui donne un nom ; il peut indiquer une URL de redirection.
3. Il copie l'endpoint affiché (`https://…/f/<identifiant>`).
4. Il colle cet endpoint dans l'attribut `action` du formulaire de son site statique.
5. Un visiteur final remplit le formulaire et l'envoie.
6. CatchForm filtre le spam, enregistre la soumission, notifie le client par e-mail et redirige le visiteur.
7. Le client consulte les soumissions dans son tableau de bord et peut les exporter en CSV.

5.3 Exigences par module

Chaque exigence est identifiée et priorisée selon la méthode **MoSCoW** : **M** = indispensable, **S** = important, **C** = souhaitable. Le statut indique l'état de réalisation.

Statut	Signification
Développé	Code écrit et couvert par les tests automatisés, tous réussis (27/27, voir chapitre 8).
Partiel	Fonction disponible sous une forme simplifiée.
À faire	Non développé ; planifié dans la feuille de route (chapitre 9).

5.3.1 Comptes et authentification (AUTH)

ID	Exigence	Prio.	Statut
AUTH-01	Inscription par e-mail et mot de passe : 8 caractères minimum, e-mail normalisé en minuscules, un seul compte par e-mail.	M	**Développé**
AUTH-02	Connexion et déconnexion, avec un message d'erreur générique en cas d'échec (ne révèle pas si le compte existe).	M	**Développé**
AUTH-03	Session mémorisée 30 jours.	S	**Développé**
AUTH-04	Retour à la page demandée après connexion, protégé contre les redirections ouvertes.	S	**Développé**
AUTH-05	Réinitialisation du mot de passe par e-mail.	M	**À faire**
AUTH-06	Vérification de l'adresse e-mail à l'inscription.	M	**À faire**
AUTH-07	Espace « Mon compte » : modification de l'e-mail et du mot de passe.	S	**À faire**

ID	Exigence	Prio.	Statut
AUTH-08	Suppression du compte et de toutes les données associées (droit à l'effacement).	M	**À faire**
AUTH-09	Connexion avec un compte Google ou GitHub.	C	**À faire**

5.3.2 Gestion des formulaires (FORM)

ID	Exigence	Prio.	Statut
FORM-01	Création d'un formulaire : nom et URL de redirection optionnelle.	M	**Développé**
FORM-02	Identifiant public aléatoire et non devinable (11 caractères) formant l'endpoint `/f/<id>`.	M	**Développé**
FORM-03	Liste des formulaires avec nombre de soumissions et date de création.	M	**Développé**
FORM-04	Paramètres modifiables : nom, redirection, activation des notifications.	S	**Développé**
FORM-05	Suppression d'un formulaire et de ses soumissions, après confirmation.	M	**Développé**
FORM-06	Validation de l'URL de redirection : http(s) uniquement, 500 caractères maximum.	M	**Développé**
FORM-07	Affichage de l'endpoint avec bouton « Copier » et exemple de code HTML prêt à coller.	S	**Développé**
FORM-08	Restriction des domaines autorisés à envoyer vers un formulaire.	C	**À faire**
FORM-09	Mise en pause d'un formulaire.	C	**À faire**

5.3.3 Réception des soumissions — endpoint public (ING)

ID	Exigence	Prio.	Statut
ING-01	Réception en POST des données d'un formulaire HTML classique (form-data ou urlencoded).	M	**Développé**
ING-02	Réception en JSON via `fetch()`, avec réponse JSON `{"ok": true}`.	M	**Développé**
ING-03	En-têtes CORS autorisant l'envoi depuis n'importe quel domaine, requêtes préliminaires (OPTIONS) comprises.	M	**Développé**
ING-04	Anti-spam par champ piège `_gotcha` : l'envoi est ignoré sans que le robot le sache.	M	**Développé**
ING-05	Champs spéciaux : `_subject` fixe l'objet de l'e-mail ; les champs commençant par « _ » ne sont pas enregistrés.	S	**Développé**
ING-06	Regroupement des champs à valeurs multiples (cases à cocher).	S	**Développé**
ING-07	Après envoi : redirection (code 303) vers l'URL configurée, sinon page « Merci » avec lien de retour.	M	**Développé**
ING-08	Erreurs explicites en HTML ou en JSON : formulaire inconnu (404), contenu illisible (400), formulaire vide (422), quota atteint (402).	M	**Développé**
ING-09	Page d'information si l'endpoint est ouvert directement dans un navigateur.	C	**Développé**
ING-10	Taille maximale d'une requête : 100 Ko.	M	**Développé**
ING-11	Limitation de débit par adresse IP et par formulaire.	M	**À faire**
ING-12	Anti-spam avancé optionnel (Cloudflare Turnstile ou reCAPTCHA).	S	**À faire**

ID	Exigence	Prio.	Statut
ING-13	Pièces jointes (fichiers).	C	**À faire**

5.3.4 Tableau de bord et données (DASH)

ID	Exigence	Prio.	Statut
DASH-01	Consultation des soumissions, de la plus récente à la plus ancienne, par pages de 50.	M	**Développé**
DASH-02	Suppression d'une soumission.	S	**Développé**
DASH-03	Export CSV compatible Excel : UTF-8 avec BOM, une colonne par champ rencontré, formules neutralisées.	M	**Développé**
DASH-04	Jauge de consommation mensuelle du plan gratuit et alerte quand le quota est atteint.	S	**Développé**
DASH-05	Recherche et filtres dans les soumissions.	C	**À faire**
DASH-06	Statistiques : envois par jour, taux de spam.	C	**À faire**
DASH-07	Dates affichées dans le fuseau horaire de l'utilisateur (aujourd'hui affichées en UTC).	C	**Partiel**

5.3.5 Notifications (NOT)

ID	Exigence	Prio.	Statut
NOT-01	E-mail au propriétaire à chaque soumission (API Resend), activable formulaire par formulaire.	S	**Développé**
NOT-02	Adresse de réponse égale à l'e-mail du visiteur : « Répondre » lui écrit directement.	S	**Développé**
NOT-03	Contenu de l'e-mail échappé (protection contre l'injection HTML).	M	**Développé**
NOT-04	Envoi en arrière-plan : n'allonge pas le temps de réponse au visiteur.	S	**Développé**
NOT-05	File d'attente avec nouvelles tentatives en cas d'échec d'envoi.	S	**À faire**
NOT-06	Destinataires multiples et accusé de réception envoyé au visiteur.	C	**À faire**
NOT-07	Webhooks et intégrations (Slack, Google Sheets, Zapier).	C	**À faire**

5.3.6 Offres et facturation (PLAN)

ID	Exigence	Prio.	Statut
PLAN-01	Plan Gratuit limité à 100 soumissions par mois, tous formulaires confondus (valeur configurable).	M	**Développé**
PLAN-02	Plan Pro sans limite et sans mention « Propulsé par CatchForm » sur la page de remerciement.	M	**Développé**
PLAN-03	Page Tarifs ; bouton « Passer Pro » qui ouvre un e-mail (solution temporaire).	S	**Développé**
PLAN-04	Commande d'administration `set-plan` pour changer le plan d'un client.	S	**Développé**
PLAN-05	Paiement en ligne Stripe (Checkout + webhooks) et activation automatique du plan Pro.	M	**À faire**
PLAN-06	Portail client : factures, changement de carte, résiliation.	M	**À faire**

ID	Exigence	Prio.	Statut
PLAN-07	E-mails d'alerte à 80 % et 100 % du quota gratuit.	S	**À faire**
PLAN-08	Offre Agence : comptes d'équipe, gestion multi-clients.	C	**À faire**

5.3.7 Site public et exploitation (PUB, ADM)

ID	Exigence	Prio.	Statut
PUB-01	Page d'accueil commerciale : proposition de valeur, étapes, fonctionnalités, appel à l'action.	M	**Développé**
PUB-02	Page Tarifs.	M	**Développé**
PUB-03	Pages d'erreur personnalisées (page introuvable, session expirée).	S	**Développé**
PUB-04	Mentions légales, CGU/CGV et politique de confidentialité.	M	**À faire**
PUB-05	Documentation utilisateur par outil (Hugo, Jekyll, Astro, GitHub Pages…).	S	**À faire**
PUB-06	Blog et tutoriels pour l'acquisition (SEO).	C	**À faire**
ADM-01	Sonde de santé `/healthz` (application et base de données).	M	**Développé**
ADM-02	Back-office : comptes, plans, volumes, modération.	S	**À faire**
ADM-03	Suivi des erreurs (Sentry) et surveillance de disponibilité.	S	**À faire**

5.4 Récapitulatif

Indicateur	Valeur
Exigences spécifiées	62
Développées	37
Partielles	1
À faire	24
Exigences indispensables (M) restant à réaliser	7

6. Exigences non fonctionnelles

6.1 Sécurité

Exigence	Statut
Mots de passe hachés et salés (Werkzeug).	Développé
Protection CSRF sur toutes les actions du tableau de bord (Flask-WTF) ; endpoint public exempté par conception.	Développé
Cookies de session HttpOnly et SameSite=Lax, réservés au HTTPS en production.	Développé
Isolation des données : un client ne peut ni voir ni modifier les données d'un autre (réponse 404).	Développé
Protection contre les redirections ouvertes après connexion.	Développé
Échappement systématique des contenus (pages, e-mails) ; formules neutralisées dans le CSV.	Développé
Taille maximale des requêtes (100 Ko) ; prise en compte du proxy HTTPS de l'hébergeur.	Développé
En-têtes de sécurité HTTP (CSP, HSTS, X-Frame-Options).	À faire
Limitation de débit sur la connexion et l'endpoint public.	À faire

6.2 Performance et disponibilité

Exigence	Statut
Réponse de l'endpoint en moins de 500 ms, hors réveil du serveur (objectif).	Non mesuré
Compteurs calculés en base de données, sans charger les soumissions en mémoire.	Développé
Pagination des soumissions (50 par page).	Développé
Disponibilité de 99,5 % en production (impossible sur le plan gratuit, qui se met en veille).	À faire
Sauvegardes quotidiennes de la base de données (offre payante).	À faire

6.3 Conformité RGPD

CatchForm stocke les données personnelles des visiteurs finaux pour le compte de ses clients. Au sens du RGPD, **CatchForm est sous-traitant** et ses clients sont responsables de traitement.

Exigence	Statut
Hébergement dans l'Union européenne (région Francfort dans `render.yaml`, conforme à la spécification Render).	Prêt
Accord de sous-traitance (DPA) intégré aux CGU ; registre des traitements.	À faire
Durée de conservation des adresses IP et des soumissions définie et documentée.	À faire
Droit à l'effacement : suppression des soumissions et des formulaires (fait), suppression du compte (à faire).	Partiel
Droit à la portabilité : export CSV par formulaire.	Développé
Polices de caractères auto-hébergées (aujourd'hui chargées depuis Google Fonts).	À faire

6.4 Ergonomie, compatibilité et maintenabilité

Exigence	Statut
Interface responsive : mobile, tablette, ordinateur.	**Développé**
Fonctionnement sans JavaScript sur le site du client (envoi HTML natif).	**Développé**
Accessibilité : libellés de formulaires présents ; audit des contrastes et de la navigation au clavier à mener.	**Partiel**
Code organisé en modules (blueprints), configuration par variables d'environnement.	**Développé**
Tests automatisés : 20 fonctions de test, 27 cas, tous réussis.	**Réussi**
Migrations de schéma de base de données (Flask-Migrate).	**À faire**
Intégration continue : tests à chaque push.	**À faire**

7. Architecture technique

7.1 Choix technologiques

Couche	Technologie	Justification
Langage	Python 3.10 ou plus (3.13 en local)	Langage déjà utilisé par le porteur du projet.
Framework web	Flask 3	Léger, éprouvé, adapté à un MVP.
Accès aux données	Flask-SQLAlchemy 3 / SQLAlchemy 2	Même code pour SQLite et PostgreSQL.
Authentification	Flask-Login	Sessions et « se souvenir de moi ».
Protection CSRF	Flask-WTF	Jetons sur toutes les actions du tableau de bord.
Base de données	SQLite (développement), PostgreSQL (production, pilote psycopg 3)	Aucune installation en local ; base robuste en production.
Serveur d'application	Gunicorn (1 processus, 8 fils d'exécution)	Standard de production pour Flask.
E-mails	API HTTP Resend	Simple, offre gratuite pour démarrer.
Hébergement	Render (Blueprint `render.yaml`), région Francfort	Déploiement en un clic, HTTPS inclus, données dans l'UE.
Interface	Gabarits Jinja, CSS sur mesure, polices Inter et JetBrains Mono	Aucune dépendance front-end à maintenir.
Tests	pytest	Tests de bout en bout avec le client de test Flask.

7.2 Schéma de fonctionnement



N°	Échange	Description
1	Site statique → CatchForm	Le visiteur final envoie le formulaire (HTML natif ou JSON via `fetch()`) vers `/f/<id>`.
2	CatchForm → PostgreSQL	Vérification du quota, puis enregistrement de la soumission.
3	CatchForm → Resend	Notification e-mail du client, envoyée en arrière-plan.
4	CatchForm → visiteur final	Redirection 303, page « Merci » ou réponse JSON.
5	Client ↔ CatchForm	Tableau de bord authentifié (session et jeton CSRF).
6	Render → CatchForm	Contrôle de santé régulier sur `/healthz`.

7.3 Modèle de données

Trois tables, reliées par des clés étrangères avec suppression en cascade : un **utilisateur** possède plusieurs **formulaires**, et un formulaire reçoit plusieurs **soumissions**.

Table users — comptes clients

Champ	Type	Contraintes	Description
id	Entier	Clé primaire	Identifiant interne
email	Texte (255)	Unique, indexé	Adresse e-mail en minuscules
password_hash	Texte (255)	Obligatoire	Empreinte du mot de passe
plan	Texte (20)	Défaut : free	Plan : free ou pro
created_at	Date-heure	UTC	Date d'inscription

Table forms — formulaires

Champ	Type	Contraintes	Description
id	Entier	Clé primaire	Identifiant interne
public_id	Texte (16)	Unique, indexé	Identifiant public de l'endpoint `/f/<public_id>`
name	Texte (120)	Obligatoire	Nom donné par le client
redirect_url	Texte (500)	Optionnel	Page de destination après envoi
notify	Booléen	Défaut : vrai	Notification e-mail activée
user_id	Entier	Clé étrangère, indexé	Propriétaire du formulaire
created_at	Date-heure	UTC	Date de création

Table submissions — soumissions

Champ	Type	Contraintes	Description
id	Entier	Clé primaire	Identifiant interne
form_id	Entier	Clé étrangère, indexé	Formulaire concerné
data	Texte	JSON	Champs envoyés par le visiteur final
ip	Texte (64)	Optionnel	Adresse IP de l'expéditeur (durée de conservation à définir)
created_at	Date-heure	UTC, indexé	Date de réception (sert au calcul du quota)

7.4 Routes de l'application

Méthode	URL	Accès	Rôle
GET	/	Public	Accueil (redirige vers le tableau de bord si connecté)
GET	/pricing	Public	Tarifs
GET	/healthz	Public	Contrôle de santé

Méthode	URL	Accès	Rôle
GET, POST	/register	Public	Inscription
GET, POST	/login	Public	Connexion
GET	/logout	Connecté	Déconnexion
GET	/dashboard	Connecté	Liste des formulaires et jauge de quota
POST	/forms	Connecté	Création d'un formulaire
GET	/forms/<id>	Propriétaire	Endpoint, paramètres et soumissions
POST	/forms/<id>/settings	Propriétaire	Mise à jour des paramètres
POST	/forms/<id>/delete	Propriétaire	Suppression du formulaire
GET	/forms/<id>/export.csv	Propriétaire	Export CSV
POST	/submissions/<n>/delete	Propriétaire	Suppression d'une soumission
GET	/f/<id>	Public	Page d'information sur l'endpoint
POST	/f/<id>	Public (CORS)	Réception d'une soumission
OPTIONS	/f/<id>	Public (CORS)	Requête préliminaire des navigateurs

7.5 Configuration

Toute la configuration passe par des variables d'environnement ; aucun secret n'est écrit dans le code.

Variable	Rôle	Valeur par défaut
SECRET_KEY	Signature des sessions (obligatoire en production, générée par Render)	Valeur de développement
DATABASE_URL	URL de la base PostgreSQL	SQLite local
SESSION_COOKIE_SECURE	Cookies réservés au HTTPS	0 (1 en production)
RESEND_API_KEY	Active les notifications e-mail	Vide : désactivées
MAIL_FROM	Expéditeur des e-mails	CatchForm <onboarding@resend.dev>
CONTACT_EMAIL	Destinataire des demandes « Passer Pro »	contact@example.com
FREE_PLAN_MONTHLY_LIMIT	Quota mensuel du plan gratuit	100

7.6 Organisation du code

Fichier ou dossier	Rôle
app/__init__.py	Fabrique d'application, pages d'erreur, commande `set-plan`
app/models.py	Modèles User, Form, Submission ; calcul du quota mensuel
app/auth.py	Inscription, connexion, déconnexion
app/dashboard.py	Tableau de bord, paramètres, suppressions, export CSV
app/ingest.py	Endpoint public `/f/<id>` : le cœur du produit
app/notify.py	Notifications e-mail via Resend

Fichier ou dossier	Rôle
app/public.py	Accueil, tarifs, `/healthz`
app/extensions.py, app/utils.py	Extensions Flask, fonctions partagées
app/templates/	9 pages HTML (gabarits Jinja)
app/static/style.css	Feuille de style responsive
config.py	Configuration par variables d'environnement
wsgi.py	Point d'entrée, en local et en production
tests/test_app.py	20 tests automatisés
render.yaml, Procfile	Déploiement sur Render ou sur un autre hébergeur
README.md, DEPLOIEMENT.md	Documentation du projet et guide de mise en ligne
exemple/contact.html	Site statique de démonstration

8. Bilan : ce qui a été réalisé

8.1 Déroulé des travaux

1. **Livrable préalable** : un tableau HTML stylé (`tableau.html`), première demande de la session, hors projet CatchForm.
2. **Étude de l'environnement** : un projet Flask existait déjà (`App.py`, Python 3.13). Il n'a **pas été modifié** ; CatchForm a été créé dans un dossier dédié, `catchform/`.
3. **Choix de l'idée** : backend de formulaires pour sites statiques, sur un marché prouvé, réalisable intégralement en MVP.
4. **Développement du serveur** : modèles de données, authentification, tableau de bord, endpoint public, notifications e-mail, quota mensuel.
5. **Durcissement de la sécurité** : CSRF, redirections ouvertes, injection de formules dans le CSV, isolation des comptes, cookies sécurisés, taille maximale des requêtes, prise en compte du proxy HTTPS.
6. **Interface** : 9 pages (accueil, tarifs, inscription, connexion, tableau de bord, détail d'un formulaire, page de remerciement, erreurs) et une feuille de style responsive.
7. **Tests automatisés** : 20 fonctions de test (27 cas) couvrant tous les parcours.
8. **Industrialisation** : point d'entrée Gunicorn, dépendances, `Procfile`, Blueprint Render, `.gitignore`.
9. **Documentation** : `README.md` (idée, modèle économique, installation), `DEPLOIEMENT.md` (mise en ligne pas à pas), page de démonstration et présent cahier des charges.
10. **Validation** : installation des dépendances, exécution des tests automatisés (27/27 réussis), recette sur un serveur réellement lancé (29/29 contrôles réussis), vérification de `render.yaml` et des limites du plan gratuit dans la documentation officielle de Render.

> **Contrainte rencontrée, puis levée**
>
> Pendant la majeure partie de la session, l'outil d'exécution de commandes est resté indisponible : le code a d'abord été écrit et relu sans pouvoir être exécuté.
>
> Après le passage en mode « Demander les permissions », les tests et la recette ont pu être exécutés. **Tout a réussi au premier lancement, sans aucune correction.**
>
> **Non vérifié à ce stade :** le rendu visuel dans un navigateur (l'outil d'aperçu est resté indisponible), l'envoi réel d'un e-mail (il faut une clé Resend) et le comportement en production.

8.2 Livrables

Livrable	Emplacement	Statut
Application web	`catchform/app/` (9 modules Python)	**Développé**
Interface	`app/templates/` (9 pages), `app/static/style.css`	**Développé**
Tests automatisés	`tests/test_app.py` (27 cas)	**Réussi (27/27)**
Script de recette	`outils_docs/recette_http.py` (29 contrôles)	**Réussi (29/29)**
Configuration de déploiement	`render.yaml`, `Procfile`, `wsgi.py`, `requirements.txt`	**Prêt (syntaxe vérifiée)**
Documentation	`README.md`, `DEPLOIEMENT.md`	**Rédigé**
Démonstration	`exemple/contact.html`	**Prêt**

Livrable	Emplacement	Statut
Cahier des charges	`docs/CatchForm_Cahier_des_charges.docx`	**Ce document**

8.3 Couverture des tests

Tous les tests sont dans `tests/test_app.py`. Ils utilisent une base SQLite en mémoire et le client de test de Flask.

Résultat : 27 cas réussis sur 27 (pytest 9.1, Python 3.13.14, le 16/10/2026, en 15 secondes environ). Les 19 avertissements affichés viennent de la bibliothèque Flask-Login (fonction de date obsolète) et non du code de CatchForm.

Test	Ce qui est vérifié
`test_public_pages`	Pages publiques (accueil, tarifs, `/healthz`) et page introuvable (404).
`test_register_login_logout`	Inscription, connexion, déconnexion, retour à la page demandée, refus d'un mauvais mot de passe.
`test_register_validation`	Refus d'un mot de passe trop court et d'un e-mail déjà utilisé, sans tenir compte des majuscules.
`test_safe_next_blocks_open_redirects`	8 cas : les redirections vers un site externe sont bloquées (URL complète, //, barre oblique inverse, tabulation).
`test_submission_flow`	Envoi HTML complet : champs enregistrés, champs « _ » ignorés, cases multiples regroupées, affichage dans le tableau de bord.
`test_json_submission_and_cors`	Envoi JSON, réponse `{"ok": true}`, en-têtes CORS et requête préliminaire OPTIONS.
`test_honeypot_silently_drops_spam`	Champ piège rempli : rien n'est enregistré, mais la réponse indique un succès.
`test_rejects_empty_invalid_and_unknown`	Erreurs 422 (vide), 400 (JSON invalide), 404 (formulaire inconnu) ; page d'information en GET.
`test_redirect_after_submission`	Redirection 303 vers l'URL configurée.
`test_invalid_redirect_url_is_rejected`	Refus d'une URL de redirection dangereuse (`javascript:`).
`test_free_plan_monthly_limit`	Quota mensuel : refus 402 au-delà, alerte au tableau de bord, remise à zéro au changement de mois, plan Pro illimité.
`test_pro_thank_you_page_has_no_branding`	Mention « Propulsé par CatchForm » présente en Gratuit, absente en Pro.
`test_email_notification`	Destinataire, objet nettoyé, adresse de réponse, échappement HTML, désactivation par formulaire.
`test_users_cannot_touch_other_users_forms`	Isolation : un autre compte obtient 404 en consultation, export et suppression.

Test	Ce qui est vérifié
`test_update_settings`	Mise à jour du nom, de la redirection et des notifications.
`test_delete_submission_and_form`	Suppression d'une soumission, puis d'un formulaire avec ses soumissions.
`test_pagination`	Pagination des soumissions : ordre et découpage en pages.
`test_csv_export_is_excel_safe`	Export CSV : BOM UTF-8, colonnes, accents, neutralisation des formules.
`test_csrf_protects_dashboard_but_not_public_endpoint`	Protection CSRF active sur le tableau de bord ; endpoint public accessible depuis un site tiers.
`test_cli_set_plan`	Commande d'administration de changement de plan.

8.4 Recette sur serveur réel

En complément des tests automatisés, l'application a été lancée sur le poste de développement et interrogée en HTTP, comme le feraient un navigateur et un site tiers. Le script `outils_docs/recette_http.py` joue le parcours complet d'un client et d'un visiteur final.

Domaine	Contrôles effectués	Résultat
Pages publiques	Accueil, `/healthz`, tarifs, feuille de style, page 404 personnalisée.	**5 / 5**
Comptes	Inscription sans jeton CSRF refusée ; inscription et arrivée sur le tableau de bord ; jauge à 0/100 ; tableau de bord protégé après déconnexion.	**4 / 4**
Formulaires	Création d'un formulaire ; affichage de son endpoint.	**2 / 2**
Réception depuis un site tiers	Page « Merci » ; lien de retour vers le site d'origine ; mention « Propulsé par CatchForm » ; en-tête CORS ; envoi JSON ; requête préliminaire OPTIONS ; robot piégé ; formulaire vide refusé (422) ; formulaire inconnu (404) ; page d'information en GET.	**10 / 10**
Consultation et export	2 soumissions enregistrées (le robot est ignoré) ; accents et cases multiples affichés ; champs internes non enregistrés ; CSV avec BOM UTF-8 ; colonnes CSV correctes.	**5 / 5**
Redirection	Redirection 303 vers la page configurée dans les paramètres.	**1 / 1**
Isolation des comptes	Un autre compte ne peut ni consulter ni exporter le formulaire (404).	**2 / 2**
Total		**29 / 29**

8.5 Limites connues du code actuel

- **Notifications sans file d'attente** : envoyées dans un fil d'exécution ; un e-mail est perdu si le serveur redémarre pendant l'envoi, et il n'y a pas de nouvelle tentative.
- **Pas de migrations** : `db.create_all()` crée les tables manquantes mais ne modifie jamais une table existante. Flask-Migrate est indispensable avant toute évolution des modèles.
- **Pas de limitation de débit** : l'endpoint public peut être visé par du spam massif.
- **E-mail non vérifié** : un tiers pourrait créer un compte avec l'adresse d'autrui et lui faire recevoir des notifications.

- **Quota approximatif sous forte concurrence** : deux envois simultanés peuvent dépasser la limite d'une unité (impact négligeable).
- **Dates en UTC** dans le tableau de bord.
- **Google Fonts** : le chargement des polices transmet l'adresse IP des utilisateurs à Google (point RGPD).
- **Hébergement gratuit** : chez Render, le service s'endort après 15 minutes sans trafic (réveil d'environ une minute) ; la base gratuite expire 30 jours après sa création, avec 14 jours de grâce pour passer à une offre payante ; une seule base gratuite par espace de travail (source : render.com/docs/free).
- **Point positif confirmé** : les services gratuits de Render bloquent les ports d'envoi d'e-mails (SMTP). L'envoi par l'API web de Resend, retenu dès la conception, n'est pas concerné.

9. Reste à faire — feuille de route

Les efforts sont exprimés en **jours de travail estimés pour une personne** (développement, tests et documentation compris). Ce sont des ordres de grandeur.

Vue d'ensemble

Phase	Objectif	Période	Effort estimé
Phase 0 — Validation technique	Prouver que le code fonctionne	Réalisée (16/10/2026)	0,25 j restant
Phase 1 — Mise en ligne de la bêta	Service public gratuit en HTTPS	Semaine 1	1,6 j
Phase 2 — Prérequis au lancement commercial	Facturer légalement et en sécurité	Semaines 2 à 4	15 j
Phase 3 — Croissance	Augmenter la valeur et la conversion	Mois 2 à 6	25,25 j

9.1 Phase 0 — Validation technique

Phase réalisée le 16/10/2026, à l'exception de la vérification visuelle.

Tâche	Résultat	Prio.	Effort	Statut
Exécuter les tests automatisés	27 cas réussis sur 27.	M	0,25 j	**Fait**
Corriger les anomalies	Aucune anomalie détectée.	M	—	**Fait**
Recette sur serveur réel	29 contrôles réussis sur 29 (`outils_docs/recette_http.py`).	M	0,5 j	**Fait**
Vérifier la configuration Render	`render.yaml` conforme à la spécification Blueprint ; limites du plan gratuit confirmées.	M	0,25 j	**Fait**
Vérification visuelle	Parcourir les pages dans un navigateur, sur ordinateur et sur mobile.	S	0,25 j	**À faire**

9.2 Phase 1 — Mise en ligne de la bêta

Tâche	Détail	Réf.	Prio.	Effort
Créer le dépôt GitHub privé	Initialiser Git et pousser le code (commandes dans `DEPLOIEMENT.md`).	—	M	0,25 j
Déployer avec le Blueprint Render	Création automatique de l'application et de la base PostgreSQL à Francfort.	—	M	0,25 j
Contrôler la production	`/healthz`, création de compte, envoi réel depuis la page de démonstration.	—	M	0,25 j
Activer les e-mails	Compte Resend et clé API dans les variables d'environnement Render.	NOT-01	S	0,25 j
Nom de domaine	Achat, CNAME vers Render, SPF/DKIM pour Resend, variable `MAIL_FROM`.	—	S	0,5 j
Contact commercial	Variable `CONTACT_EMAIL` pour le bouton « Passer Pro ».	PLAN-03	S	0,1 j

9.3 Phase 2 — Prérequis au lancement commercial

Tâche	Détail	Réf.	Prio.	Effort
Paiement Stripe	Stripe Checkout, webhooks, activation et désactivation automatiques du plan Pro.	PLAN-05	M	3 j
Portail client	Factures, changement de carte, résiliation (portail Stripe).	PLAN-06	M	1 j
Réinitialisation du mot de passe	Lien à usage unique et à durée limitée, envoyé par e-mail.	AUTH-05	M	1 j
Vérification de l'e-mail	Confirmation obligatoire avant tout envoi de notification.	AUTH-06	M	1 j
Espace « Mon compte »	Changement d'e-mail et de mot de passe, suppression du compte et des données.	AUTH-07/08	M	1,5 j
Limitation de débit	Flask-Limiter sur la connexion et sur l'endpoint public.	ING-11	M	0,5 j
Migrations de base de données	Flask-Migrate (Alembic), avant toute évolution des modèles.	—	M	0,5 j
En-têtes de sécurité HTTP	CSP, HSTS, X-Frame-Options, Referrer-Policy.	—	S	0,5 j
Polices auto-hébergées	Supprimer l'appel à Google Fonts (transfert d'adresses IP hors UE).	—	S	0,25 j
Alertes de quota	E-mail à 80 % et à 100 % du quota gratuit.	PLAN-07	S	0,5 j
Intégration continue	GitHub Actions : tests à chaque push.	—	S	0,5 j
Supervision	Sentry (erreurs) et surveillance de disponibilité.	ADM-03	S	0,5 j
Pages légales	Mentions légales, CGU/CGV, politique de confidentialité.	PUB-04	M	2 j
Conformité RGPD	Accord de sous-traitance (DPA), registre des traitements, durée de conservation des IP.	—	M	1 j
Hébergement payant	Service sans mise en veille et base de données sauvegardée.	—	M	0,25 j
Statut juridique	Structure permettant de facturer (par exemple micro-entreprise).	—	M	0,5 j
Étude tarifaire	Relevé à jour des offres et des prix des concurrents.	—	S	0,5 j

9.4 Phase 3 — Croissance

Tâche	Détail	Réf.	Prio.	Effort
File d'attente des e-mails	Envoi fiable avec nouvelles tentatives (par exemple RQ et Redis).	NOT-05	S	1,5 j
Anti-spam avancé	Cloudflare Turnstile, optionnel par formulaire.	ING-12	S	1 j
Back-office d'administration	Comptes, plans, volumes, modération.	ADM-02	S	2 j
Documentation par outil	Guides Hugo, Jekyll, Astro, GitHub Pages, Webflow…	PUB-05	S	2 j
Webhooks et intégrations	Webhooks, Slack, Google Sheets, Zapier.	NOT-07	C	4 j

Tâche	Détail	Réf.	Prio.	Effort
Recherche et filtres	Dans les soumissions.	DASH-05	C	1 j
Statistiques	Envois par jour, taux de spam.	DASH-06	C	2 j
Pièces jointes	Fichiers (CV, documents) avec stockage objet.	ING-13	C	3 j
Domaines autorisés	Restreindre les sites autorisés à envoyer vers un formulaire.	FORM-08	C	0,5 j
Fuseau horaire	Dates affichées dans le fuseau de l'utilisateur.	DASH-07	C	0,5 j
Connexion Google / GitHub	Authentification OAuth.	AUTH-09	C	1,5 j
Accusé de réception	E-mail de confirmation au visiteur, destinataires multiples.	NOT-06	C	1 j
Mise en pause d'un formulaire	Refuser temporairement les envois.	FORM-09	C	0,25 j
Offre Agence	Comptes d'équipe, multi-clients, marque blanche.	PLAN-08	C	5 j
Contenus SEO	Blog et tutoriels d'acquisition.	PUB-06	C	Continu

10. Planning prévisionnel

Période	Phase	Jalons
Semaine 1	Phases 0 et 1	Tests et recette au vert (fait) ; vérification visuelle ; bêta en ligne sur Render ; e-mails actifs.
Semaines 2 et 3	Phase 2 — technique	Paiement Stripe ; comptes complets ; sécurité renforcée.
Semaine 4	Phase 2 — légal et exploitation	Pages légales ; hébergement payant ; supervision ; **lancement public**.
Mois 2 et 3	Phase 3	File d'attente des e-mails ; anti-spam avancé ; documentation ; back-office.
Mois 4 à 6	Phase 3	Intégrations ; statistiques ; offre Agence.

Planning indicatif pour une personne à temps plein sur la partie technique (la phase 2 représente environ 15 jours de travail). La semaine 1 commence à la validation de ce document.

Jalons de décision

- **Fin de la phase 0** : le code est-il fonctionnel ? Sinon, corriger avant toute mise en ligne.
- **Fin de la phase 1** : la bêta fonctionne-t-elle en conditions réelles ? Recueillir les premiers retours.
- **Avant le lancement public** : toutes les exigences « M » de la phase 2 sont-elles réalisées, notamment le paiement, les pages légales et l'hébergement payant ?
- **À 3 et 6 mois** : comparer les indicateurs aux objectifs du chapitre 2.4 et ajuster la stratégie.

11. Recette — critères d'acceptation du MVP

Le MVP est accepté lorsque tous les scénarios ci-dessous donnent le résultat attendu, en local puis en production.

N°	Scénario	Résultat attendu	Statut
1	Créer un compte	Redirection vers le tableau de bord, message de bienvenue	**Réussi en local**
2	Créer un formulaire	Endpoint affiché, en `https://` en production	**Réussi en local**
3	Envoyer le formulaire de démonstration	Page « Merci » ; soumission visible dans le tableau de bord	**Réussi en local**
4	Envoyer avec une URL de redirection configurée	Arrivée sur la page configurée	**Réussi en local**
5	Envoyer en JSON via `fetch()`	Réponse `{"ok": true}`, sans quitter la page	**Réussi en local**
6	Remplir le champ piège `_gotcha`	Rien n'est enregistré ; page « Merci » affichée	**Réussi en local**
7	Dépasser le quota du plan gratuit	Refus (402) ; jauge rouge et alerte dans le tableau de bord	**Réussi (test auto)**
8	Exporter en CSV et ouvrir dans Excel	Colonnes correctes, accents lisibles	**Partiel**
9	Ouvrir le formulaire d'un autre compte	Page introuvable (404)	**Réussi en local**
10	Recevoir une notification e-mail	E-mail reçu ; « Répondre » écrit au visiteur	**À tester**
11	Ouvrir `/healthz`	`{"status": "ok"}`	**Réussi en local**
12	Utiliser le service sur mobile	Interface lisible et utilisable	**À tester**

« Réussi en local » : vérifié le 16/10/2026 sur un serveur lancé sur le poste de développement (chapitre 8.4), à rejouer en production. « Réussi (test auto) » : couvert par un test automatisé. « Partiel » : encodage et colonnes du CSV vérifiés, ouverture dans Excel encore à faire. Les scénarios 10 et 12 demandent une clé Resend et un contrôle visuel.

12. Budget prévisionnel

> Les tarifs ci-dessous sont des **ordres de grandeur** à vérifier sur les sites des fournisseurs au moment de la souscription.

12.1 Phase bêta

Poste	Solution	Coût indicatif
Application web	Render, offre gratuite (mise en veille après inactivité)	0 €
Base de données	Render Postgres, offre gratuite (durée de vie limitée)	0 €
E-mails	Resend, offre gratuite	0 €
Nom de domaine (optionnel)	Registrar (OVH, Gandi, Cloudflare…)	≈ 10 à 15 € / an
Total		**≈ 0 €** (hors domaine)

12.2 Production

Poste	Solution	Coût indicatif	Source à consulter
Application web	Render, offre Starter	≈ 7 $ / mois	render.com/pricing
Base de données	Render Postgres, offre Basic 256 Mo	≈ 6 à 7 $ / mois	render.com/pricing
E-mails	Resend	Gratuit jusqu'à un certain volume, puis payant	resend.com/pricing
Paiement	Stripe	Commission par transaction, sans abonnement	stripe.com/fr/pricing
Suivi des erreurs	Sentry, offre gratuite	0 €	sentry.io/pricing
Nom de domaine	Registrar	≈ 10 à 15 € / an	—
Total		**≈ 15 $ / mois** + domaine	

12.3 Coûts hors technique

- **Statut juridique** : création d'une micro-entreprise (démarche gratuite en France), à confirmer selon la situation du porteur.
- **Documents juridiques** : modèles adaptés, ou relecture par un professionnel pour les CGU/CGV et le DPA.
- **Comptabilité et assurance** : selon le statut choisi ; une responsabilité civile professionnelle est recommandée.

13. Analyse des risques

Risque	Probabilité	Impact	Mesure de réduction
Spam massif ou abus de l'endpoint public	Élevée	Moyen	Limitation de débit, Turnstile, quotas
Perte de données (base gratuite expirée, absence de sauvegarde)	Moyenne	Élevé	Base payante avec sauvegardes dès le premier client
Non-conformité RGPD	Moyenne	Élevé	Pages légales, DPA, durées de conservation, hébergement UE, polices auto-hébergées
Notifications classées en spam	Moyenne	Moyen	Domaine vérifié (SPF, DKIM) chez Resend
Concurrence d'acteurs établis	Élevée	Moyen	Niche francophone et européenne, prix, qualité du support
Adoption trop faible	Moyenne	Élevé	Référencement, communautés, démarchage des freelances
Lenteur due à la mise en veille (plan gratuit)	Élevée	Moyen	Passage à l'hébergement payant avant le lancement
Dépendance aux fournisseurs (Render, Resend, Stripe)	Faible	Moyen	Code portable (Procfile, standards ouverts)
Anomalies non détectées en production	Faible	Élevé	Tests (27/27) et recette (29/29) réussis en local ; recette à rejouer en production ; intégration continue

14. Glossaire

Terme	Définition
Backend	Partie serveur d'une application, qui reçoit, traite et stocke les données.
Site statique	Site composé de fichiers HTML, CSS et JavaScript servis tels quels, sans traitement côté serveur.
Endpoint	Adresse web qui reçoit des données. Chez CatchForm : `/f/<identifiant>`.
SaaS	Logiciel en tant que service : application utilisée en ligne, sur abonnement.
MVP	Produit minimum viable : première version réduite aux fonctions essentielles.
MRR	Revenu mensuel récurrent (Monthly Recurring Revenue).
Freemium	Modèle combinant une offre gratuite limitée et une offre payante.
Champ piège (honeypot)	Champ invisible pour les humains, rempli par les robots, qui permet d'ignorer le spam.
CORS	Mécanisme des navigateurs qui autorise un site à envoyer des requêtes vers un autre domaine.
CSRF	Attaque qui fait exécuter une action à un utilisateur connecté à son insu ; contrée par un jeton secret.
Redirection ouverte	Faille permettant d'envoyer un utilisateur vers un site malveillant via un lien d'apparence légitime.
RGPD	Règlement général sur la protection des données de l'Union européenne.
DPA	Accord de sous-traitance des données, exigé par l'article 28 du RGPD.
Blueprint Render	Fichier `render.yaml` décrivant l'infrastructure que Render crée automatiquement.
Webhook	Requête envoyée automatiquement à un autre service lors d'un événement.
MoSCoW	Méthode de priorisation : Must (indispensable), Should (important), Could (souhaitable), Won't (exclu).

Annexes

A. Code d'intégration type

Code que le client colle sur son site statique, en remplaçant l'adresse par celle de son formulaire :

```
<form method="POST" action="https://votre-app.onrender.com/f/IDENTIFIANT">
  <input name="email" type="email" required>
  <textarea name="message"></textarea>
  <input type="hidden" name="_subject" value="Nouveau message du site">
  <input type="text" name="_gotcha" style="display:none">
  <button>Envoyer</button>
</form>
```

Champ spécial	Effet
`_gotcha`	Champ piège : si ce champ caché est rempli, l'envoi est ignoré.
`_subject`	Objet de l'e-mail de notification.
`email`	Devient l'adresse de réponse de l'e-mail de notification.

B. Commandes utiles (Windows, PowerShell)

Installation et lancement en local

```
cd C:\Users\toshiba\Flask\catchform
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements-dev.txt
.venv\Scripts\python wsgi.py
```

L'application est alors disponible sur `http://127.0.0.1:5000`.

Exécution des tests

```
.venv\Scripts\python -m pytest
```

Recette sur serveur réel (serveur lancé au préalable)

```
python ..\outils_docs\recette_http.py
```

Passage d'un client au plan Pro

```
.venv\Scripts\flask --app wsgi set-plan client@exemple.com pro
```

Mise à jour de la production

```
git add .
git commit -m "Description du changement"
git push
```

Render redéploie automatiquement l'application à chaque `git push`. La procédure complète de première mise en ligne figure dans `DEPLOIEMENT.md`.